SOFTWARE DESIGN DESCRIPTOR DOCUMENT (SDD)

For

IMDBSE2

TBD

FIRST SEM, 2025

Prepared By
De Guzman, Asher Nathaniel E. -
Dela Cruz, Rupert C. -
Macaraeg, Mark Cyrus P. -

Contribution Sign-off

Document Sign-Off	
DEVELOPERS	**REVIEWER** **Graeven Yuri B. Videz** Database 2 Adviser
De Guzman, Asher Nathaniel E. Developer	
Dela Cruz, Rupert C. Developer	
Macaraeg, Mark Cyrus P. Developer	

Version History

Version #	Implemented By	Revision Date	Approved By	Approved Date	Reason / Description
1.0	Cooked Group	02/27			Completed the first draft of the SDD

Table of Contents

1. INTRODUCTION..... 5
1.1 PURPOSE..... 5
1.2 SCOPE..... 5
1.3 PROPOSED METHODOLOGY..... 5
1.4 DEFINITIONS AND ACRONYMS..... 5
1.5 REFERENCES..... 5
2. DATA DESIGN..... 5
2.1 DATA DESCRIPTION & ENTITY-RELATIONSHIP DIAGRAM..... 5
2.1.a Data Design Approach..... 5
2.1.b Access Control Domain..... 5
2.1.c Academic Management Domain..... 5
2.1.d Engagement and Gamification Domain..... 6
2.1.e Moderation and Community Contribution Domain..... 6
2.1.f Notification Domain..... 6
2.1.g Entity Relationship Summary..... 6
2.2 SYSTEM ARCHITECTURE..... 7
2.2.a Presentation Layer (Client-Side)..... 7
2.2.b Application Layer (Server-Side)..... 7
2.2.c Data Layer..... 8
2.2.d External Services Layer..... 8
2.2.e Architectural Flow Summary..... 8
2.2.f Security Considerations at Architectural Level..... 9
2.3 LOGICAL ARCHITECTURE..... 9
2.4 DATA DICTIONARY..... 9

List of Tables and Figures

Diagram 1.0 Relational Flows Table..... 8
Figure 2.0 Architectural Flow Diagram..... 10
Table 3.0.0 Users Database Table Schema..... 13
Table 3.0.1 Roles Database Table Schema..... 13
Table 3.0.2 User_Roles Database Table Schema..... 14
Table 3.0.3 Permissions Database Table Schema..... 14
Table 3.0.4 Role_Permissions Database Table Schema..... 14
Table 3.1.0 Courses Database Table Schema..... 14
Table 3.1.1 User_crouses Database Table Schema..... 15
Table 3.1.2 Modules Database Table Schema..... 15
Table 3.1.3 Difficulty-Roles Database Table Schema..... 16
Table 3.1.4 Assignments Database Table Schema..... 16
Table 3.1.5 Submissions Database Table Schema..... 17
Table 3.2.0 Mini_Game_Templates Database Table Schema..... 17
Table 3.2.1 Mini_Game_Attempts Database Table Schema..... 18
Table 3.2.2 XP_Transactions Database Table Schema..... 18
Table 3.2.3 Levels Database Table Schema..... 18
Table 3.2.4 User_Levels Database Table Schema..... 19
Table 3.3.0 Challenges Database Table Schema..... 19
Table 3.3.1 Notifications Database Table Schema..... 20

1. INTRODUCTION

1.1 PURPOSE

The purpose of this Software Design Descriptor (SDD) document is to provide a comprehensive architectural and logical blueprint for Kody: Gamified Programming Learning Platform with Course Management System (K:GPLPCMS). While the Software Requirements Specification (SRS) defined what the system must do, this SDD details *how* the system will be built to meet those requirements. It serves as the primary technical reference for the development team, outlining the data design, relational entity structures, system architecture, and modular logic necessary to construct the platform.

1.2 SCOPE

Kody is a web-based educational technology platform designed to integrate traditional Learning Management System (LMS) functions with interactive coding execution and gamified progression mechanisms. The scope of this design encompasses the architectural mapping of the following core system features:

- Role-Based Access Control (RBAC) governing Administrators, Instructors, Learners, Contributors, and Moderators.
- Data structures for course creation, module organization, and assignment management.
- An integrated in-browser coding execution engine with automated test-case validation.
- A hybrid evaluation framework combining automated coding results with manual grading and feedback.
- Gamification logic and entities supporting XP accumulation, ranking levels, and interactive mini-games.
- A unified performance dashboard consolidating academic and gamified metrics.
- Moderation workflows for contributor-submitted challenges.

1.3 PROPOSED METHODOLOGY

The development and implementation of the system design will proceed through a structured, phased methodology to ensure controlled deployment and scope management.

- Phase 1: Requirements and Policy Consolidation: Formalization of role-based access control, academic vs. gamified logic separation, and validation requirements.
- Phase 2: Interface Modeling and Visual Structuring: Visualization of user interfaces, dashboard layouts, and interaction flows for submissions.
- Phase 3: Core Functional Implementation: Development of the authentication layer, course management modules, coding validation engine, and XP progression mechanisms.
- Phase 4: Hosting and Deployment Preparation: Configuration of the cloud hosting environment and integration with external authentication and communication APIs.
- Phase 5: Testing and Quality Assurance: Functional testing of the validation workflows, XP calculations, and security access controls
- Phase 6: Maintenance and Iterative Enhancement: Performance optimization, bug resolution, and future expansion of supported programming languages.

1.4 DEFINITIONS AND ACRONYMS

Definitions:

- Coding Execution Engine: A system component responsible for compiling and executing user-submitted code against predefined test cases.

- Contributor: A user role authorized to submit coding challenges or supplemental learning materials for review.
- Gamification: The application of game design elements (e.g., XP, ranks, progress indicators) in non-game contexts to enhance engagement.
- Moderator: A user role responsible for reviewing contributed content and enforcing platform guidelines.
- Unified Performance Dashboard: An interface consolidating academic grades, coding results, and progression metrics in a single view.

Acronyms:

- **API:** Application Programming Interface; protocols for integrating external services.
- **LMS:** Learning Management System; software used for course distribution, assignment management, and grading.
- **RBAC:** Role-Based Access Control; a formal system model restricting access based on assigned user roles.
- **SRS:** Software Requirements Specification; a document describing system requirements.

1.5 REFERENCES

De Guzman, A. N. E., dela Cruz, R. C., & Macaraeg, M. C. P. (2026). Software Requirements Specification (SRS) K:GPLPCMS. University of Baguio.

De Guzman, A. N. E., dela Cruz, R. C., & Macaraeg, M. C. P. (2026). Gamified Programming Learning Platform with Integrated Course Management System (System Profile V1.4). University of Baguio.

2. DATA DESIGN

2.1 DATA DESCRIPTION & ENTITY-RELATIONSHIP DIAGRAM

2.1.a Data Design Approach

K:GPLPCMS utilizes a relational SQL-based database to ensure structured storage, referential integrity, and transactional consistency across academic, engagement, and access control processes.

The database design follows Third Normal Form (3NF) principles to:

- Minimize redundancy
- Preserve data integrity
- Enforce relational constraints
- Maintain separation between academic and gamified workflows

Primary and foreign key constraints are used to maintain referential integrity between entities.

The system database is logically divided into four functional domains:

1. Access Control Domain
2. Academic Management Domain
3. Engagement & Gamification Domain
4. Moderation and Communication Domain

2.1.b Access Control Domain

This domain implements Role-Based Access Control (RBAC) and governs user authentication and authorization.

Core Entities:

1. users – Stores registered user information including identity, login credentials, and status.
2. roles – Defines system roles (Administrator, Instructor, Learner, Contributor, Moderator).
3. user_roles – Junction table supporting many-to-many relationship between users and roles.

4. permissions – Defines system-level permissions.
5. role_permissions – Junction table mapping permissions to roles.

Relationship Structure:

- A user may possess multiple roles.
- A role may be assigned to multiple users.
- Roles aggregate multiple permissions.
- Permissions can be reused across roles.

2.1.c Academic Management Domain

This domain governs structured course delivery and assignment workflows.

Core Entities:

1. courses – Represents instructor-created academic courses.
2. user_courses – Junction table recording learner enrollment and course completion status.
3. modules – Logical subdivisions of courses.
4. assignments – Coding-based academic tasks associated with modules.
5. submissions – Stores learner assignment submissions including validation status and manual grading.

Relationship Structure:

- One course contains multiple modules.
- One module contains multiple assignments.
- One assignment may receive multiple submissions.
- One learner may enroll in multiple courses.
- One learner may submit multiple assignment solutions.

Academic grading is separated from XP tracking to preserve integrity between assessment and gamified progression.

2.1.d Engagement and Gamification Domain

This domain manages XP accumulation, rank progression, and mini-game activities.

Core Entities:

1. difficulty_rules – Defines difficulty levels and associated XP values.
2. mini_game_templates - System-defined mini-game formats.
3. mini_game_attempts – Records user attempts within mini-game templates.
4. xp_transactions – Records all XP awards in a unified ledger format.
5. levels – Defines rank thresholds based on XP accumulation.
6. user_levels – Tracks historical level achievements per user.

Relationship Structure:

- Assignments and mini-games both generate XP entries in xp_transactions.
- XP accumulation is calculated by aggregating xp_transactions per user.
- Levels define XP thresholds required for rank advancement.
- user_levels maintains progression history.

The unified XP transaction model ensures a single progression pool while maintaining traceability of XP sources.

2.1.e Moderation and Community Contribution Domain

This domain manages contributor-submitted coding challenges and content governance.

Core Entities:

1. challenges – Stores contributor-submitted coding problems.

Each challenge includes review status (Pending, Approved, Rejected). Moderation actions are recorded via reviewer reference fields. This structure supports controlled publication of community content while preserving administrative oversight.

2.1.f Notification Domain

Core Entity:

1. notifications – Stores system-generated messages related to role approvals, submissions, moderation outcomes, and general updates.

Each notification is tied to a specific user and includes read/unread tracking.

2.1.g Entity Relationship Summary

The overall database design establishes the following high-level relational flows:

Course Completion	Content Contribution
A. 1. enrolls in Course 2. completes Assignments 3. submits Submissions 4. earns XP Transactions 5. progresses through Levels B. 1. participates in Mini-Game Attempts 2. earns XP Transactions	1. Contributors submits Challenges 2. Challenges are reviewed by Moderator

Diagram 1.0 Relational Flows Table

This relational structure enforces separation between (1) academic grading logic, (2) engagement mechanics, (3)access governance, and (4) moderation workflows, while maintaining unified identity management.

2.2 SYSTEM ARCHITECTURE

K:GPLPCMS follows a layered web-based client–server architecture deployed within a cloud-hosted environment.

The system consists of four primary architectural layers:

1. Presentation Layer
2. Application Layer
3. Data Layer
4. External Services Layer

These layers interact through secure HTTPS communication channels and RESTful API interfaces.

2.2.a Presentation Layer (Client-Side)

The Presentation Layer consists of the web-based graphical user interface accessed via modern desktop and mobile web browsers.

Responsibilities:

- Display role-specific dashboards
- Render course, module, and assignment views
- Provide in-browser code editor interface
- Display mini-game interfaces
- Present unified performance dashboard
- Send user actions to backend services via HTTP requests

This layer does not directly access the database. All data retrieval and validation requests are routed through the Application Layer.

2.2.b Application Layer (Server-Side)

The Application Layer contains the core business logic of the platform.

Primary Responsibilities:

- User authentication handling (OAuth integration)
- Role-Based Access Control (RBAC) enforcement
- Course and module management logic

- Assignment validation coordination
- Mini-game logic execution
- XP calculation and transaction recording
- Level threshold evaluation
- Moderation workflow processing
- Notification generation
- API communication with external services

This layer acts as an intermediary between the Presentation Layer and the Data Layer.

All database operations occur through controlled server-side logic to prevent unauthorized direct data access.

2.2.c Data Layer

The Data Layer consists of a relational SQL database responsible for persistent storage.
Primary Responsibilities:

- Store user data and role assignments
- Maintain academic records (courses, modules, assignments, submissions)
- Store mini-game attempts
- Record XP transactions
- Track level progression
- Store moderation records
- Maintain notification logs

The database enforces referential integrity using primary and foreign key constraints.

No client-side component interacts directly with the database.

2.2.d External Services Layer

K:GPLPCMS integrates with selected third-party services to support core functionality

1. Google Authentication Service

- Used for secure user identity verification through OAuth 2.0 protocol.

User → Google OAuth → Application Server → Database session creation.

Role elevation beyond authentication is handled internally.

2. Code Execution API

- Used for runtime validation of supported programming languages (e.g., Python, Java, C++, JavaScript, PHP).

Application Server → Execution API → Validation Result → Application Server → Database update.

For non-runtime languages (e.g., HTML, SQL), validation logic is handled internally within the Application Layer.

3. Email Service Provider

- Role approval notifications
- Assignment alerts
- System updates

Triggered by backend logic.

4. Cloud Hosting Infrastructure

The entire system is deployed within a cloud-hosted environment providing:

- Web server hosting

- Database hosting
- Secure HTTPS communication
- Scalable resource allocation

2.2.e Architectural Flow Summary

The high-level interaction flow is as follows:

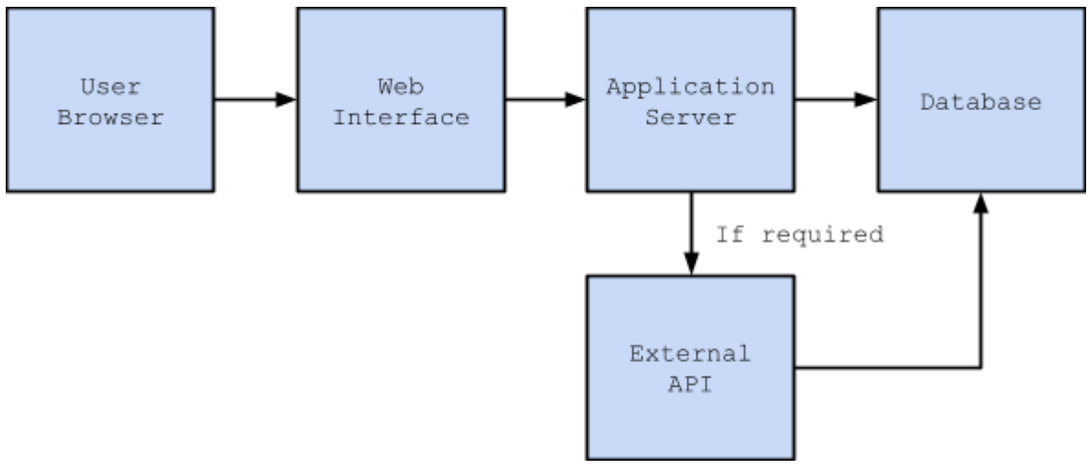

Figure 2.0 Architectural Flow Diagram

This structure ensures:

- Separation of concerns
- Security isolation
- Scalable deployment
- Controlled validation logic
- Clear separation between academic and engagement subsystems

2.2.f Security Considerations at Architectural Level

- All communications occur over HTTPS.
- Authentication tokens are validated server-side.
- Code execution occurs outside the core application server via sandboxed third-party service.
- Role checks are enforced at middleware level before accessing protected routes.

2.3 LOGICAL ARCHITECTURE

2.3.a Architectural Pattern

K:GPLPCMS follows a modular layered architecture within the Application Layer, separating concerns into independent logical components.

The internal logic is organized into the following major subsystems:

1. Authentication & Access Control Module
2. Academic Management Module
3. Validation & Execution Module
4. Engagement & Progression Module
5. Mini-Game Engine Module
6. Moderation Module
7. Notification Module
8. Data Access Layer

Each module operates independently but communicates through defined service interfaces.

2.3.b Authentication & Access Control Module

Responsibilities:

- Handle OAuth authentication callback
- Manage session creation
- Assign system roles
- Enforce Role-Based Access Control (RBAC)
- Validate permissions before route execution

Internal Components:

- OAuth Handler

- Role Resolver
- Permission Middleware
- Session Validator
- This module ensures that all user actions are authorized before reaching business logic components.

2.3.c Academic Management Module

Responsibilities:

- Course creation and modification
- Module structuring
- Assignment creation
- Enrollment management
- Grade recording
- Academic progress tracking

Internal Components:

- Course Service
- Module Service
- Assignment Service
- Enrollment Service
- Gradebook Service

This module strictly manages academic data and does not interact with XP calculations directly.

2.3.d Validation & Execution Module

Responsibilities:

- Route coding submissions to appropriate validation method
- Integrate with external execution API
- Handle structured validation for HTML and SQL
- Return validation status to Academic Module
- Trigger XP award events upon successful validation

Internal Components:

- Execution API Adapter
- Structured Validator
- Result Processor

This module acts as a validation gateway and isolates external code execution from core business logic.

2.3.e Engagement & Progression Module

Responsibilities:

- Calculate total XP
- Record XP transactions
- Evaluate level thresholds
- Update user rank
- Provide leaderboard data

Internal Components:

- XP Service
- Level Evaluation Service
- Leaderboard Service

This module operates independently from grading logic and relies solely on XP transaction aggregation.

2.3.f Mini-Game Engine Module

Responsibilities:

- Load system-defined mini-game templates
- Inject problem content
- Process mini-game submissions
- Validate mini-game results
- Award XP upon success

Internal Components:

- Template Loader
- Mini-Game Validator

- XP Award Trigger

Mini-game logic is isolated from academic assignments to preserve grading integrity.

2.3.g Moderation Module

Responsibilities:

- Receive contributor-submitted challenges
- Review challenge content
- Approve or reject submissions
- Publish approved challenges

Internal Components:

- Challenge Submission Handler
- Review Processor
- Publication Controller

This module ensures that only moderated content becomes accessible to learners.

2.3.h Notification Module

Responsibilities:

- Generate system alerts
- Send email notifications
- Log in-platform notifications
- Track read/unread status

Internal Components:

- Notification Generator
- Email Dispatcher
- Notification Repository

This module responds to events triggered by other modules (e.g., role approval, submission review).

2.3.i Data Access Layer (DAL)

The Data Access Layer abstracts direct database interaction from business logic.

Responsibilities:

- Execute SQL queries
- Manage transactions
- Enforce data consistency
- Provide repository interfaces for modules

This ensures that:

- Business logic remains independent of SQL syntax
- Future database migration remains possible
- Security policies are centralized

2.3.j Logical Interaction Flow

A typical academic submission flow:

```
User → Authentication Module
→ Academic Module (Submit Assignment)
→ Validation Module (Execute Code)
→ Academic Module (Record Status)
→ Engagement Module (Award XP)
→ Notification Module (Optional Feedback)
```

A typical mini-game flow:

```
User → Mini-Game Engine
→ Mini-Game Validator
→ Engagement Module (Award XP)
→ Leaderboard Update
```

A moderation flow:

```
Contributor → Moderation Module (Submit Challenge)
Moderator → Review Processor
```

→ Publication Controller

2.3.k Logical Separation Principles

The logical architecture enforces:

- Separation between academic grading and gamified progression
- Isolation of external API dependencies
- Modular scalability for future expansion
- Controlled privilege enforcement via RBAC middleware

This ensures maintainability, extensibility, and security alignment with the requirements defined in the SRS

2.4 DATA DICTIONARY

2.4.a Access Control Domain

TABLE NAME: users				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
user_id	INT	Unique identifier for each registered user	Primary Key	AUTO_INCREMENT
first_name	VARCHAR(100)	User’s given name		NOT NULL
last_name	VARCHAR(100)	User’s family name		NOT NULL
email	VARCHAR(150)	Unique email address used for authentication	Unique	NOT NULL
password_hash	VARCHAR(255)	Encrypted password hash (if not exclusively OAuth-based)		NOT NULL
status	ENUM('Active','Inactive')	Indicates whether account is usable		Default 'Active'
date_registered	DATETIME	Timestamp of account creation		DEFAULT CURRENT_TIMESTAMP

Table 3.0.0 Users Database Table Schema

TABLE NAME: roles				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
role_id	INT	Unique identifier for system role	Primary Key	AUTO_INCREMENT
role_name	VARCHAR(100)	Name of role (Admin, Instructor, Learner, etc.)	Unique	NOT NULL
description	TEXT	Description of role responsibilities		NULL allowed

Table 3.0.1 Roles Database Table Schema

TABLE NAME: user_roles				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
user_id	INT	References user	Composite PK / FK	ON DELETE CASCADE
role_id	INT	References role	Composite PK / FK	ON DELETE CASCADE
Note: Composite Primary Key: (user_id, role_id)				

Table 3.0.2 User_Roles Database Table Schema

TABLE NAME: permissions				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
permission_id	INT	Unique identifier for permission	Primary Key	AUTO_INCREMENT
permission_name	VARCHAR(100)	System permission name	Unique	NOT NULL
description	TEXT	Description of permission capability		NULL allowed

Table 3.0.3 Permissions Database Table Schema

TABLE NAME: role_permissions				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
role_id	INT	References role	Composite PK / FK	ON DELETE CASCADE
permission_id	INT	References permission	Composite PK / FK	ON DELETE CASCADE

Table 3.0.4 Role_Permissions Database Table Schema

2.4.b Academic Management Domain

TABLE NAME: courses				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
course_id	INT	Unique identifier for course	Primary Key	AUTO_INCREMENT
course_name	VARCHAR(150)	Title of the course		NOT NULL
description	TEXT	Course overview and objectives		NULL allowed
status	ENUM('Active','Archived')	Indicates course availability		Default 'Active'
created_by	INT	User who created the course	Foreign Key	ON DELETE SET NULL
created_at	DATETIME	Timestamp of course creation		DEFAULT CURRENT_TIMESTAMP

Table 3.1.0 Courses Database Table Schema

TABLE NAME: user_courses

Field Name	Data Type	Description	Key	Constraints
user_id	INT	Enrolled user	Composite PK / FK	ON DELETE CASCADE
course_id	INT	Course enrolled in	Composite PK / FK	ON DELETE CASCADE
enrollment_date	DATETIME	Enrollment timestamp		DEFAULT CURRENT_TIMESTAMP
status	ENUM('Active','Dropped','Completed')	Enrollment state		Default 'Active'
Note: Composite Primary Key: (user_id, course_id)				

Table 3.1.1 User_crouses Database Table Schema

TABLE NAME: modules				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
dule_id	INT	Unique identifier for module	Primary Key	AUTO_INCREMENT
course_id	INT	Parent course	Foreign Key	ON DELETE CASCADE
title	VARCHAR(150)	Module title		NOT NULL
status	ENUM('Draft','Approved','Archived')	Module publication state		Default 'Draft'
created_by	INT	Instructor who created module	Foreign Key	ON DELETE CASCADE
approved_by	INT	Moderator/Admin who approved module	Foreign Key	ON DELETE SET NULL
approved_at	DATETIME	Approval timestamp		NULL allowed
Relationships: course_id → courses(course_id) created_by → users(user_id) approved_by → users(user_id)				

Table 3.1.2 Modules Database Table Schema

TABLE NAME: difficulty_rules				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
difficulty_id	INT	Unique identifier for difficulty level	Primary Key	AUTO_INCREMENT
difficulty_name	VARCHAR(50)	Difficulty label (Easy, Medium, Hard)	Unique	NOT NULL
xp_value	INT	XP awarded for completion		NOT NULL

estimated_time_minutes	INT	Estimated completion time		NULL allowed

Table 3.1.3 Difficulty-Roles Database Table Schema

TABLE NAME: assignments				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
assignment_id	INT	Unique identifier for assignment	Primary Key	AUTO_INCREMENT
module_id	INT	Parent module	Foreign Key	ON DELETE CASCADE
title	VARCHAR(150)	Assignment title		NOT NULL
problem_statement	TEXT	Assignment instructions		NOT NULL
difficulty_id	INT	Associated difficulty level	Foreign Key	NOT NULL
created_by	INT	Instructor creator	Foreign Key	ON DELETE CASCADE
status	ENUM('Active','Archived')	Assignment availability		Default 'Active'
created_at	DATETIME	Timestamp of creation		DEFAULT CURRENT_TIMESTAMP
Relationships: module_id → modules(module_id) difficulty_id → difficulty_rules(difficulty_id) created_by → users(user_id)				

Table 3.1.4 Assignments Database Table Schema

TABLE NAME: submissions				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
submission_id	INT	Unique identifier for submission	Primary Key	AUTO_INCREMENT
user_id	INT	Learner who submitted	Foreign Key	ON DELETE CASCADE
assignment_id	INT	Assignment submitted	Foreign Key	ON DELETE CASCADE
submitted_code	TEXT	Source code submitted		NOT NULL
submission_time	DATETIME	Timestamp of submission		DEFAULT CURRENT_TIMESTAMP
validation_status	ENUM('Pending','Correct','Incorrect')	Automated validation result		Default 'Pending'
manual_grade	INT	Instructor-assigned grade		NULL allowed

reviewed_by	INT	Instructor reviewer	Foreign Key	ON DELETE SET NULL
Relationships: user_id → users(user_id) assignment_id → assignments(assignment_id) reviewed_by → users(user_id)				

Table 3.1.5 Submissions Database Table Schema

2.4.c Engagement & Gamification Domain

TABLE NAME: mini_game_templates				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
template_id	INT	Unique identifier for mini-game template	Primary Key	AUTO_INCREMENT
template_name	VARCHAR(150)	Name of predefined mini-game format		NOT NULL
description	TEXT	Description of mini-game mechanics		NULL allowed
difficulty_id	INT	Associated difficulty level	Foreign Key	NOT NULL
status	ENUM('Active','Inactive')	Indicates availability		Default 'Active'
Relationships: difficulty_id → difficulty_rules(difficulty_id)				

Table 3.2.0 Mini_Game_Templates Database Table Schema

TABLE NAME: mini_game_attempts				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
ttempt_id	INT	Unique identifier for attempt	Primary Key	AUTO_INCREMENT
user_id	INT	Learner attempting mini-game	Foreign Key	ON DELETE CASCADE
template_id	INT	Mini-game template used	Foreign Key	ON DELETE CASCADE
problem_statement	TEXT	Injected problem instance	—	NOT NULL
submitted_code	TEXT	Learner’s submitted solution	—	NOT NULL
result_status	ENUM('Correct','Incorrect')	Validation result	—	NOT NULL
xp_awarded	INT	XP granted for this attempt	—	NOT NULL

attempt_time	DATETIME	Timestamp of attempt	—	DEFAULT CURRENT_TIMESTAMP
Relationships: user_id → users(user_id template_id → mini_game_templates(template_id				

Table 3.2.1 Mini_Game_Attempts Database Table Schema

TABLE NAME: xp_transactions				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
xp_transaction_id	INT	Unique identifier for XP entry	Primary Key	AUTO_INCREMENT
user_id	INT	User receiving XP	Foreign Key	ON DELETE CASCADE
source_type	ENUM('Assignment','MiniGame')	XP origin classification	—	NOT NULL
source_id	INT	ID of assignment or mini-game attempt	—	NOT NULL
xp_amount	INT	XP awarded	—	NOT NULL
date_awarded	DATETIME	Award timestamp	—	DEFAULT CURRENT_TIMESTAMP
reason	VARCHAR(255)	Optional explanation	—	NULL allowed
Relationships: user_id → users(user_id)				

Table 3.2.2 XP_Transactions Database Table Schema

TABLE NAME: levels				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
level_id	INT	Unique identifier for level	Primary Key	AUTO_INCREMENT
level_number	INT	Sequential level number	Unique	NOT NULL
xp_threshold	INT	Minimum XP required	—	NOT NULL
milestone_reward	VARCHAR(255)	Optional achievement description	—	NULL allowed

Table 3.2.3 Levels Database Table Schema

TABLE NAME: user_levels				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
user_level_id	INT	Unique identifier for record	Primary Key	AUTO_INCREMENT

user_id	INT	User achieving level	Foreign Key	ON DELETE CASCADE
level_id	INT	Level achieved	Foreign Key	ON DELETE CASCADE
achieved_at	DATETIME	Timestamp of level attainment		DEFAULT CURRENT_TIMESTAMP
Unique Constraint: user_id, level_id Relationships: user_id → users(user_id) level_id → levels(level_id)				

Table 3.2.4 User_Levels Database Table Schema

2.4.d Moderation & Communication Domain

TABLE NAME: challenges				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
challenge_id	INT	Unique identifier for challenge	Primary Key	AUTO_INCREMENT
title	VARCHAR(150)	Challenge title	—	NOT NULL
problem_statement	TEXT	Coding problem description	—	NOT NULL
difficulty_id	INT	Associated difficulty level	Foreign Key	NOT NULL
submitted_by	INT	Contributor who submitted challenge	Foreign Key	NOT NULL
status	ENUM('Pending','Approved','Rejected')	Moderation state	—	Default 'Pending'
reviewed_by	INT	Moderator who reviewed challenge	Foreign Key	ON DELETE SET NULL
reviewed_at	DATETIME	Timestamp of moderation action	—	NULL allowed
Relationships: difficulty_id → difficulty_rules(difficulty_id) submitted_by → users(user_id) reviewed_by → users(user_id)				

Table 3.3.0 Challenges Database Table Schema

TABLE NAME: notifications				
Field Name	**Data Type**	**Description**	**Key**	**Constraints**
notification_id	INT	Unique identifier for notification	Primary Key	AUTO_INCREMENT
user_id	INT	Recipient of notification	Foreign Key	ON DELETE CASCADE
message	TEXT	Notification content	—	NOT NULL

is_read	BOOLEAN	Indicates whether notification has been viewed	—	Default FALSE
created_at	DATETIME	Timestamp of creation	—	DEFAULT CURRENT_TIMESTAMP
Relationships: user_id → users(user_id)				

Table 3.3.1 Notifications Database Table Schema